
CSE 539 Course Project Report

TLS Handshake with PAKE

Hira Naseer | Arizona State University

Overview

This project illustrates a lightweight TLS 1.3-like handshake involving client-server interaction where the server is authenticated by presenting its digital certificate, and the client is authenticated using a password, without the server actually knowing the password used by the client. All of this is done using the password-authenticated key exchange method known as OPAQUE. This project is implemented in Python as two separate processes running on the localhost (127.0.0.1) and communicating with each other using sockets. All components employed in this project follow the material covered in class lectures.

System Architecture

Parties Involved

The system consists of three parties:

- **Certificate Authority (CA):** A trusted third party which signs the certificate of the server. In this project, the function of a CA is mimicked by the keys and certificates already generated before and placed into the certifications/ folder. Public key of this CA is shared among both the client and the server.
- **Server:** Listens for connection on port 127.0.0.1:5000, holds signed certificate from CA, stores registered users data, and handles handshake and encryption of messages.
- **Client:** Connects to the server, verifies its certificate and uses OPAQUE protocol to authenticate the user and establish encryption keys.

Components

The protocol composes multiple cryptographic primitives covered in the course:

Component	Purpose
Key exchange	Replaced by PAKE via OPAQUE
Digital signature	Server signs handshake messages using ECDSA with curve P-256; client verifies after obtaining the server's public key and validating the certificate
Certificates	CA signs the server's key pair; client validates the certificate
Password-based authentication	Client registers password at server during setup phase; mutual authentication during handshake via OPAQUE
OPRF	Authenticates the password without a secure communication channel; core building block of OPAQUE
HKDF	Derives handshake traffic secrets and application traffic secrets using SHA-256 as the hash function
Authenticated encryption	Protects all communication after the handshake using AES-GCM

Communication Flow

1. Client registers its password at the server during the setup phase (before the handshake)
2. Client connects and sends a client hello with a blinded password
3. Server responds with its certificate, CA certificate, and a digital signature
4. Client obtains the server's public key, validates the certificate, and verifies the signature
5. Both parties complete OPAQUE for mutual authentication and key exchange
6. Handshake traffic secrets and application traffic secrets are derived via HKDF
7. Both parties exchange and verify key confirmation tags
8. Secure communication begins protected by AES-GCM authenticated encryption

Implementation

The project is organized into four Python files, each responsible for a distinct component of the protocol.

ca_setup.py (PKI and Certificates):

This file emulates a Certificate Authority and runs once before the protocol begins. It generates a key pair for the CA using elliptic curve cryptography (curve P-256 / SECP256R1) and creates a self-signed CA certificate. It then generates a key pair for the server, registers the server's key at the CA, and issues a signed certificate. All keys and certificates are saved as *.pem* files in the *certifications/* directory. The CA's public key is made available to both the server and client at startup, allowing the client to validate the server's certificate during the handshake.

opaque.py (Password Authenticated Key Exchange):

This file implements the OPAQUE protocol and its key components. The OPRF is handled across three functions: *client_blind()* hashes the password using SHA-256 and applies a deterministic

blinding factor; *server_evaluate()* applies the server's secret key to the blinded value using HMAC-SHA256; and *client_finalize()* combines the server response with the password hash to derive the password-based key *rw*. The **OPAQUEServer** class manages user registration and login, storing per-user OPRF keys and encrypted envelopes. The **OPAQUEClient** class handles envelope encryption during registration and decryption during login using AES-GCM with *rw* as the key. Helper functions include *hkdf_derive()* for HKDF-SHA256 key derivation and *serialize_public_key()* for public key encoding.

server.py (Server Process):

The server runs on *127.0.0.1:5000* as a separate process and handles two types of requests. During **registration** (setup phase), it receives the client's blinded password, evaluates it using its secret OPRF key via HMAC-SHA256, and stores the returned encrypted envelope. During the **handshake**, it receives the client hello, sends its CA-signed certificate along with an ECDSA signature over the handshake data, performs PAKE via OPAQUE, derives handshake traffic secrets and application traffic secrets via HKDF-SHA256, and verifies key confirmation tags. Once the secure channel is established, it decrypts the client's message using AES-GCM and sends an encrypted reply.

client.py (Client Process):

The client runs as a separate process and connects to the server. During **registration**, it blinds the password using the OPRF, receives the server's HMAC-SHA256 evaluation, derives *rw*, generates a long-term key pair, encrypts it under *rw* using AES-GCM to create an envelope, and sends it to the server. During the **handshake**, it sends the blinded password and client hello, receives the server's certificate and ECDSA signature, obtains the server's public key, validates the certificate against the CA, verifies the ECDSA signature, completes PAKE via OPAQUE, derives handshake traffic secrets and application traffic secrets via HKDF-SHA256, exchanges key confirmation tags, and finally sends and receives messages protected by AES-GCM authenticated encryption.

Security Intuition

Secrecy:

All communication after the handshake is protected by AES-GCM authenticated encryption using a session key derived via HKDF-SHA256. The session key is never transmitted over the network. It is independently derived by both parties from shared material. An attacker intercepting network traffic cannot read any messages.

Integrity:

AES-GCM authenticated encryption ensures any modification to ciphertext causes decryption to fail. The server also signs its handshake messages using ECDSA with curve P-256, allowing the client to detect any tampering before the secure session begins.

Authenticity:

Server authenticity is established via its CA-signed certificate and ECDSA signature. The client obtains the server's public key, validates the certificate, and verifies the ECDSA signature before

proceeding. Client authenticity is established via PAKE (The client proves knowledge of the correct password without revealing it). The OPRF ensures the server never sees the plaintext password, and only the correct password produces the right AES-GCM decryption key rw , preventing impersonation.

Password Security

The OPAQUE protocol ensures the server never sees the client's plaintext password at any point. The OPRF allows the password to be authenticated without a secure communication channel. Even if the server's database is compromised, the attacker cannot recover the original password.

Key Confirmation

After deriving session keys via HKDF-SHA256, both parties exchange HMAC-based key confirmation tags. This ensures both sides have derived the same keys before AES-GCM encrypted communication begins, preventing man-in-the-middle and replay attacks.

Attack Resistance

- **Replay attacks:** prevented via key confirmation tags derived from the session key. A replayed message will fail verification as it cannot match the fresh session key.
- **Man-in-the-middle attacks:** prevented via the combination of CA-signed certificates, ECDSA signatures, and PAKE. An attacker cannot forge the server's certificate or impersonate the client without the correct password.
- **Password compromise:** prevented via OPRF. Even if the server's database is stolen, the attacker cannot recover the plaintext password.

Conclusion

This project successfully implements a simplified TLS 1.3 handshake with PAKE using OPAQUE. The protocol composes multiple cryptographic primitives including key exchange, ECDSA digital signatures, certificates, password-based authentication via OPRF, HKDF-SHA256 key derivation, and AES-GCM authenticated encryption to achieve secrecy, integrity, and authenticity. The client and server run as separate Python processes communicating over localhost sockets, fulfilling all project requirements.

Output

Server Terminal Output:

```
PS C:\Users\Zeeshan\cse539_project> python server.py
[SERVER] Starting server on 127.0.0.1:5000
[SERVER] Waiting for client connection...

[SERVER] Client connected from ('127.0.0.1', 57323)
[SERVER] Registration request for user: zeeshan
[SERVER-OPAQUE] Registration step 1 complete for user: zeeshan
[SERVER-OPAQUE] Registration complete for user: zeeshan
[SERVER] Registration complete for user: zeeshan ✓
[SERVER] Connection closed.

[SERVER] Client connected from ('127.0.0.1', 57324)
[SERVER] Handshake started for user: zeeshan
[SERVER-OPAQUE] Login step 1 complete for user: zeeshan
[SERVER] Certificate and signature sent to client
[SERVER-OPAQUE] HKMQV complete. Session key derived.
[SERVER] Session keys derived via HKDF ✓
[SERVER] Key confirmation successful ✓
[SERVER] Secure channel established ✓
[SERVER] Received encrypted message: 'Authentication complete. This message is end-to-end encrypted.' ✓
[SERVER] Sent encrypted reply to client ✓
[SERVER] Connection closed.
█
```

Client Terminal Output:

```
PS C:\Users\Zeeshan\cse539_project> python client.py

[CLIENT] Starting registration for user: zeeshan
[CLIENT] Registration complete for user: zeeshan ✓

[CLIENT] Connecting to server 127.0.0.1:5000
[CLIENT] Handshake started...
[CLIENT] Certificate verified - signed by CA ✓
[CLIENT] Server signature verified ✓
[CLIENT] Password authenticated via OPAQUE ✓
[CLIENT] Session keys derived via HKDF ✓
[CLIENT] Mutual authentication successful ✓
[CLIENT] Secure channel established ✓
[CLIENT] Sent encrypted message: 'Authentication complete. This message is end-to-end encrypted.' ✓
[CLIENT] Received encrypted reply: 'Session established. Encrypted channel verified successfully.' ✓
PS C:\Users\Zeeshan\cse539_project> █
```